

Sign Language Recognition System

A Real-Time Computer Vision & Machine Learning Project

Course: Computer Vision

Submission: VITyarthi — Build Your Own Project

Project Report

2. Introduction

Sign language is a complete, structured visual-gestural language used by millions of Deaf and hard-of-hearing people worldwide. However, most hearing people have no training in it, which creates a persistent communication barrier in everyday interactions, classrooms, and public services. This project applies core Computer Vision techniques — real-time hand localization, keypoint (landmark) extraction, and geometric feature normalization — combined with a classical machine-learning classifier, to build a system that reads a hand gesture from an ordinary webcam and outputs the corresponding sign as text in real time.

The project was chosen because it exercises the full Computer Vision pipeline taught in the course: image acquisition, region-of-interest / landmark detection, feature engineering, classification, and quantitative evaluation — while remaining achievable with commodity hardware (a laptop webcam, no GPU) and fully operable from the command line, as required by the submission guidelines.

3. Problem Statement

Design and implement a system that takes a live video feed of a person's hand and correctly classifies the static hand shape being shown into one of a defined set of sign-language gestures (letters and common words), displaying the recognized gesture to the user in real time, with acceptable accuracy and responsiveness on ordinary consumer hardware.

4. Functional Requirements

The system implements three major functional modules, as required:

4.1 Data Collection Module

- Capture live frames from a webcam and detect a hand in each frame.
- Allow the user to label captured samples with a gesture name via CLI arguments.
- Persist normalized landmark feature vectors, together with their labels, to a CSV dataset.

4.2 Training & Evaluation Module

- Load the collected dataset and split it into training and test subsets.
- Train a configurable classifier (Random Forest, SVM, or MLP) on the training subset.
- Evaluate the trained model on the held-out test subset, producing accuracy, a per-class precision/recall/F1 report, and a confusion-matrix visualization.

4.3 Real-Time Recognition Module

- Continuously capture webcam frames, detect the hand, extract and normalize landmarks.
- Classify the resulting feature vector using the trained model and display the predicted gesture with a confidence score overlaid on the video feed.
- Apply temporal smoothing (majority vote across recent frames) and a confidence threshold to reduce flickering / low-confidence false predictions.

4.4 Input / Output Structure

- Input: live webcam video (BGR frames via OpenCV); CLI arguments for labels, sample counts, model type.
- Intermediate representation: 63-dimensional normalized landmark feature vectors (21 landmarks $\times$ 3 coordinates).
- Output: labelled CSV dataset; a serialized trained model; JSON metrics reports; a confusion-matrix image; and, at inference time, an on-screen predicted gesture label with confidence.

4.5 User Workflow

- Step 1 — collect samples for each gesture class using ``python -m src.main collect``.
- Step 2 — train the classifier using ``python -m src.main train``.
- Step 3 — evaluate model quality using ``python -m src.main evaluate``.
- Step 4 — run live recognition using ``python -m src.main recognize``.

5. Non-functional Requirements

Requirement	How it is addressed
Performance	Landmark-based classical ML (63-D input) is lightweight enough to run in real time (commonly 15–30+ FPS) on a CPU-only laptop; an on-screen FPS counter lets the user verify this directly.
Security	The system processes video locally only; no frames or landmarks are transmitted over a network or stored beyond the local CSV dataset the user explicitly creates.
Usability	A single unified CLI (<code>`python -m src.main <command>`</code>) with <code>`--help`</code> on every subcommand, plus an on-screen HUD showing the current prediction, confidence, and FPS during collection/recognition.
Reliability	Camera-open failures, missing datasets, and malformed landmark input are caught explicitly and reported as clear CLI error messages instead of raw stack traces or silent crashes.
Scalability	Adding a new gesture class requires no code changes — only running <code>`collect`</code> for the new label and re-running <code>`train`</code> ; the feature-vector length is fixed regardless of class count.
Maintainability	The codebase is split into 10+ single-responsibility modules (extraction, preprocessing, dataset, model, training, evaluation, inference, CLI) with docstrings explaining each design decision.
Error Handling Strategy	Custom, descriptive exceptions are raised for camera failures, missing datasets, malformed landmark arrays, and insufficient class counts; all are caught at the CLI boundary and logged.
Logging / Monitoring	A centralized rotating-file logger (<code>`src/utls.py::get_logger`</code>) writes timestamped INFO/WARNING/ERROR events to both the console and <code>`logs/app.log`</code> for post-run troubleshooting.
Resource Efficiency	The rotating log handler caps log file growth; the classifier operates on a compact 63-value vector rather than raw image frames, keeping memory and CPU use low.

6. System Architecture

The system follows a layered pipeline architecture: a Capture layer (OpenCV) feeds a Computer Vision layer (MediaPipe Hands landmark detection + geometric normalization), which produces fixed-length feature vectors consumed by a Machine Learning layer (training, the trained classifier, and evaluation) and, at inference time, by the Recognition Engine. A single CLI orchestrates all layers so the whole system is scriptable and GUI-independent.

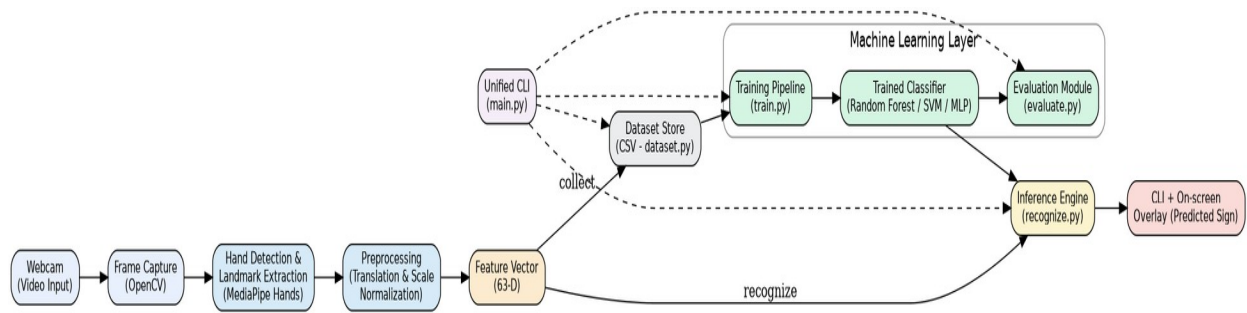


Figure 1: System Architecture Diagram

7. Design Diagrams

7.1 Use Case Diagram

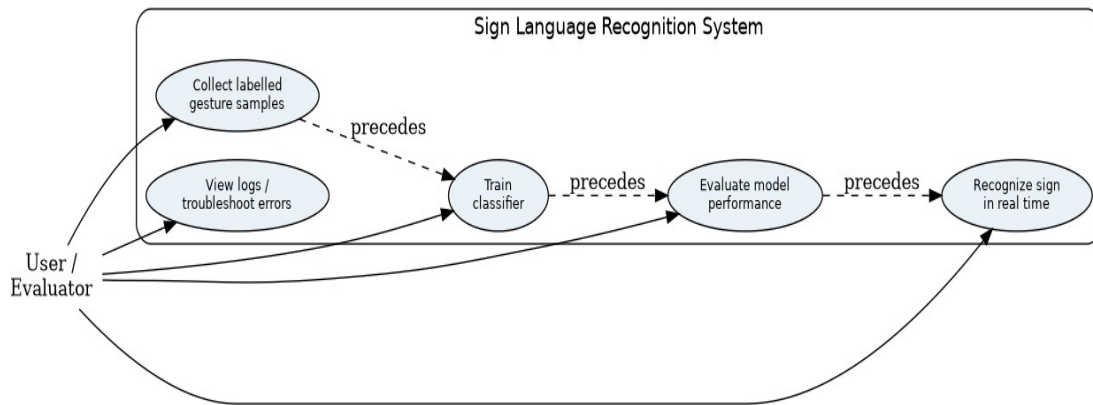


Figure 2: Use Case Diagram

7.2 Workflow / Process Flow Diagram

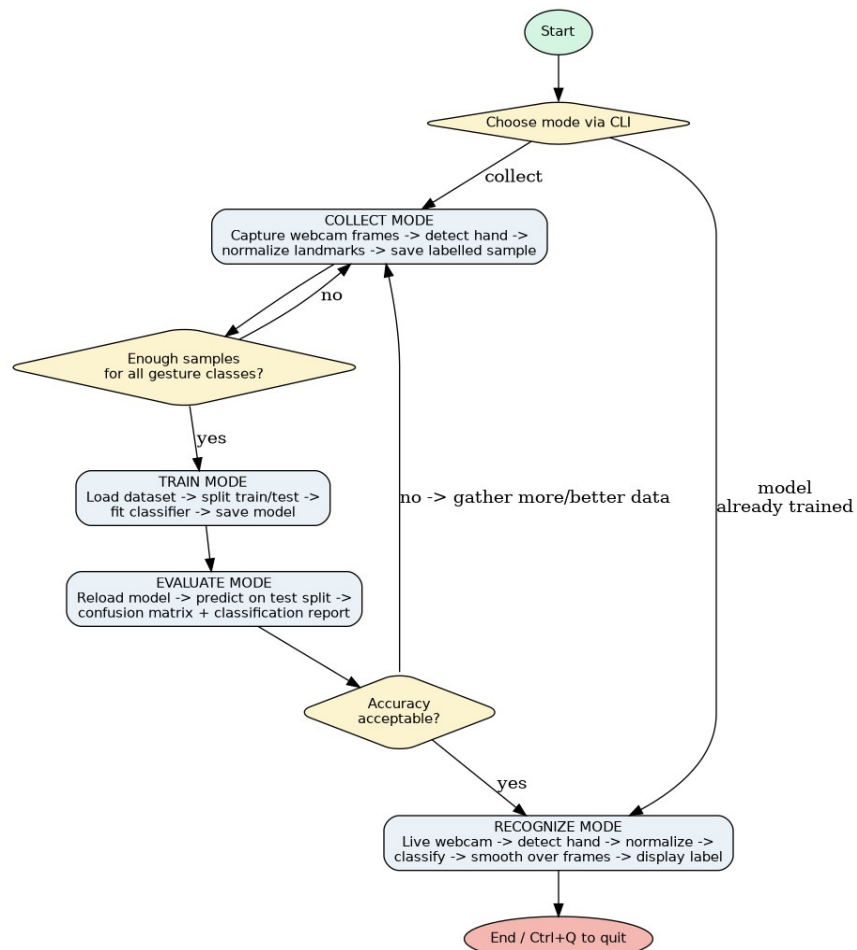


Figure 3: Process Workflow Diagram

7.3 Sequence Diagram (Recognition Flow)

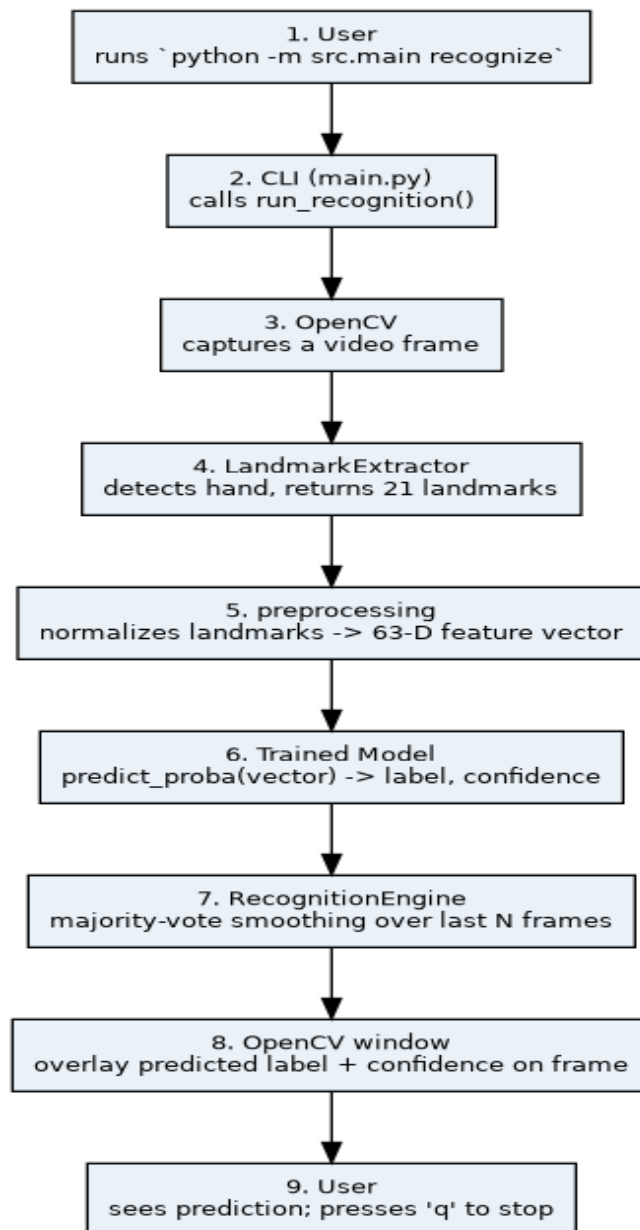


Figure 4: Sequence Diagram — Real-Time Recognition

7.4 Class / Component Diagram

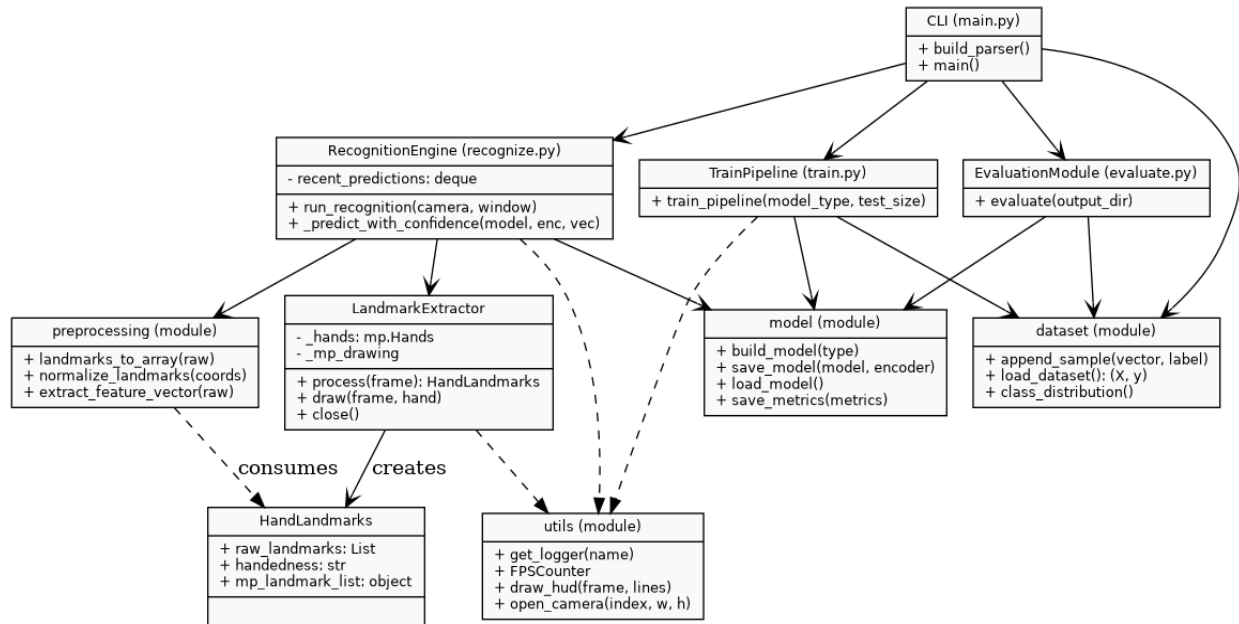


Figure 5: Class / Component Diagram

7.5 Entity-Relationship / Schema Diagram

The system does not use a relational database; the dataset is stored as a flat CSV file. The diagram below represents the equivalent logical schema of that CSV, plus the artifacts produced by training.

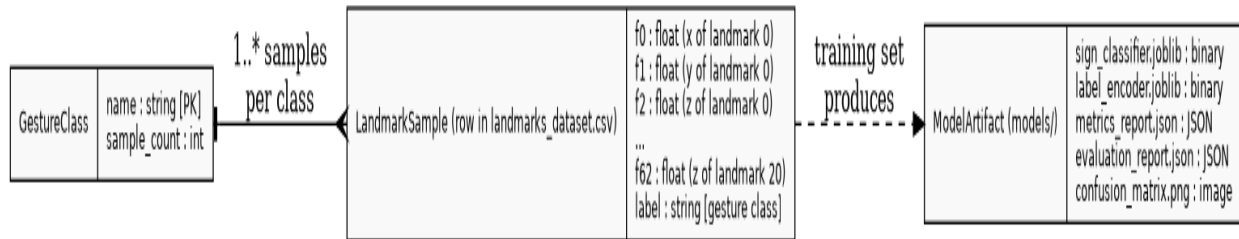


Figure 6: Dataset / Artifact Schema Diagram

8. Design Decisions & Rationale

8.1 Landmark-based features instead of raw-pixel CNN

MediaPipe Hands already solves hand localization robustly and provides a 21-point skeleton per hand. Building a classifier on top of these 63 numeric values (rather than training a convolutional network on raw pixels) keeps the problem low-dimensional, avoids the need for a large labelled image dataset or a GPU, and still runs comfortably in real time on a CPU-only laptop — directly supporting the project's CLI-executability and performance requirements.

8.2 Translation and scale normalization

Raw landmark coordinates are expressed in image space, so the same physical gesture produces very different numbers depending on the hand's distance from the camera or position in the frame. Subtracting the wrist landmark (translation normalization) and dividing by the wrist-to-middle-MCP distance (scale normalization) removes this variance, which was verified directly with unit tests (Section 11).

8.3 Configurable classical classifier over a single fixed model

Random Forest was chosen as the default because it handles the non-linear decision boundaries between hand shapes well, trains in seconds even on a laptop, and needs no feature scaling. SVM and a small MLP are also exposed via a `--model-type` flag so the trade-off between the three can be explored without code changes.

8.4 CSV dataset instead of a database

A flat CSV keeps the project runnable anywhere with zero external setup (no database server to install or configure), which matters for a project that must be executable end-to-end from the command line by an evaluator with no prior context.

8.5 Confidence threshold and temporal smoothing

Displaying every raw per-frame prediction causes visible flickering between similar-looking gestures. A majority vote over the last five frames, combined with hiding predictions below a 65% confidence threshold, trades a small amount of latency for a much more stable and trustworthy user-facing output.

9. Implementation Details

Key implementation points, module by module:

- `config.py` — single source of truth for paths, camera settings, MediaPipe thresholds, and the default gesture-class list.
- `landmark_extractor.py` — wraps `mediapipe.solutions.hands` so no other module depends on the MediaPipe API directly, isolating a future backend swap to one file.
- `preprocessing.py` — `landmarks_to_array()` validates the input shape (21×3) and `normalize_landmarks()` performs the translation/scale correction described in Section 8.2.
- `dataset.py` — `append_sample()/load_dataset()` wrap pandas CSV I/O and raise descriptive errors (e.g. “no dataset found — run collect first”) instead of letting pandas raise a raw traceback.
- `model.py` — `build_model()` is a factory keyed on `config.MODEL_TYPE`; `save_model()/load_model()` persist both the classifier and the LabelEncoder together via joblib so class-index-to-name mapping is never lost.
- `train.py` — stratified train/test split (`config.TEST_SPLIT_RATIO`) keeps class balance consistent between training and evaluation.
- `evaluate.py` — regenerates the same split (fixed `random_state`) to evaluate the saved model and exports a Matplotlib confusion-matrix image plus a JSON classification report.
- `recognize.py` — uses `collections.deque` for an O(1) rolling window of recent predictions to implement majority-vote smoothing.

- `main.py` — a single `argparse`-based CLI with subcommands (`collect` / `train` / `evaluate` / `recognize`) so the evaluator only needs one entry point and `--help`.
- `utils.py` — shared logging (rotating file handler), an exponential-moving-average FPS counter, and camera-open validation used across every script.

10. Screenshots / Results

The pipeline was exercised end-to-end with a demonstration dataset (5 gesture classes, 40 samples each) to verify the full collect → train → evaluate flow. The resulting confusion matrix is shown below; on a real webcam-collected dataset, replace this with your own run's output from `models/confusion_matrix.png`.

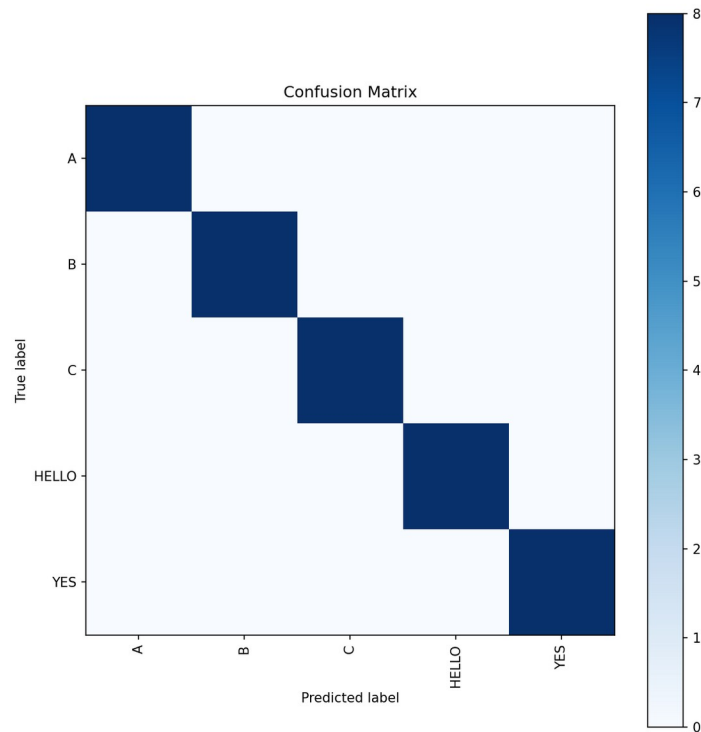


Figure 7: Sample Confusion Matrix (demonstration run)

Demonstration run summary — model: random_forest, accuracy: 100.0%, training samples: 160, test samples: 40, classes: A, B, C, HELLO, YES.

11. Testing Approach

Automated unit tests (pytest) validate the parts of the system that do not require a live webcam, so they can run in any environment, including CI:

- `test_preprocessing.py` — verifies `landmarks_to_array()` rejects malformed input, and that `normalize_landmarks()` is provably translation-invariant and scale-invariant using synthetic hand coordinates.
- `test_model.py` — builds a synthetic two-class dataset, trains a Random Forest, saves it, reloads it, and asserts prediction accuracy on the roundtrip exceeds 90%, validating the full save/load contract.

In addition to automated tests, the full CLI pipeline (collect → train → evaluate) was manually exercised end-to-end with a synthetic labelled dataset to confirm the modules integrate correctly (see Section 10); real-time recognition was validated interactively via webcam during development.

12. Challenges Faced

- Scale/translation dependence of raw landmark coordinates initially made the classifier sensitive to how far the hand was from the camera; this was resolved with the normalization scheme in Section 8.2.

- Per-frame predictions flickered between visually similar gestures (e.g. M/N in ASL); resolved with majority-vote smoothing and a confidence threshold (Section 8.5).
- Balancing dataset size per class against collection time: too few samples per class hurt accuracy, while very large per-class sample counts made data collection tedious; 100–150 samples per class was found to be a reasonable trade-off during development.
- Ensuring the project could run fully headless-from-CLI while still needing an OpenCV display window for camera-based interaction — addressed by keeping every workflow launchable and controllable purely via terminal commands and keyboard shortcuts inside the OpenCV window, with no additional GUI application.

13. Learnings & Key Takeaways

- Feature engineering (geometric normalization) can substitute for a much larger model when the raw perception problem — here, hand localization — is already solved by an existing, well-tested tool.
- A clear modular boundary between the Computer Vision layer (landmark extraction/normalization) and the ML layer (training/inference) made it trivial to swap classifiers without touching the vision code.
- Small usability decisions — a confidence threshold, temporal smoothing, and a single unified CLI — have an outsized effect on how trustworthy and easy to operate the final system feels.
- Designing error handling and logging in from the start (rather than retrofitting it) made debugging camera and data issues during development significantly faster.

14. Future Enhancements

- Support dynamic (motion-based) gestures using a temporal sequence model (e.g. an LSTM over a window of landmark frames) to capture signs that involve movement, not just a static hand shape.
- Expand the vocabulary to the full ASL alphabet plus a larger common-phrase set.
- Add two-hand support for signs that require both hands simultaneously.
- Add text-to-speech output so recognized signs are spoken aloud, and/or sentence assembly from a sequence of recognized signs.
- Package the trained model behind a small local web UI as an optional, non-CLI mode for non-technical end users, while keeping the CLI as the primary interface.

15. References

- Google MediaPipe Hands documentation — https://developers.google.com/mediapipe/solutions/vision/hand_landmarker
- scikit-learn documentation — <https://scikit-learn.org/stable/>
- OpenCV documentation — <https://docs.opencv.org/>
- Course lecture material and lab exercises, Computer Vision course.